

结构的根基

基础数据结构 · 8题 · C++ & Python 双语对照

NQ129 单链表 AcWing 826

实现一个单链表，链表初始为空，支持三种操作：(1) 向链表头插入一个数；(2) 删除第k个插入的数后面的数；(3) 在第k个插入的数后插入一个数。现在要对该链表进行M次操作，进行完所有操作后，从头到尾输出整个链表。 $1 \leq M \leq 100000$ 。所有操作保证合法。注意：题目中第k个插入的数并不是指当前链表的第k个数。例如操作过程中一共插入了n个数，则按照插入的时间顺序，这n个数依次为：第1个插入的数，第2个插入的数，...第n个插入的数。

输入

第一行包含整数M，表示操作次数。接下来M行，每行包含一个操作命令，操作命令可能为以下几种：(1) “H x”，表示向链表头插入一个数x。(2) “D k”，表示删除第k个输入的数后面的数（当k为0时，表示删除头结点）。(3) “I k x”，表示在第k个输入的数后面插入一个数x（此操作中k均大于0）。

输出

共一行，将整个链表从头到尾输出。

来源

AcWing 826

输入

```
10
H 9
I 1 1
D 1
D 0
H 6
I 3 6
I 4 5
I 4 5
I 3 4
D 6
```

输出

```
6 4 6 5
```

提示 [原题链接](#) [Y总讲解](#) [参考题解](#) [Y总代码](#)

C++

```

#include <iostream>

using namespace std;

const int N = 100010;

// head 表示头节点的下标, e[N]表示节点i的值
// ne[N]表示节点i的next指针是多少, idx存储当前已经用到了哪个节点
int head, e[N], ne[N], idx;

void init()
{
    head = -1; // 一开始设置为-1
    idx = 0; // 当前用到了0个节点
}

// 将x插入到头节点
void add_to_head(int x)
{
    e[idx] = x, ne[idx] = head, head = idx, idx++;
}

// 将x插到下标是k的点后面
void add(int k, int x)
{
    e[idx] = x, ne[idx] = ne[k], ne[k] = idx, idx++;
}

// 将下标是k的点后面的点删掉
void remove(int k)
{
    ne[k] = ne[ne[k]];
    // idx--;
}

int main()
{
    int m; cin >> m;

    init(); // 初始化

    while (m--)
    {
        int k, x;
        char op; // 操作字符

        cin >> op;
        if (op == 'H')
        {
            cin >> x;
            add_to_head(x);
        }
        else if (op == 'D')
        {
            cin >> k;
            if (!k) head = ne[head]; // 删除头节点
            else remove(k-1);
        }
        else
        {
            cin >> k >> x;
            add(k-1, x);
        }
    }

    // 打印单链表
    for (int i = head; i != -1; i = ne[i]) cout << e[i] << '

```

Python

```

# AcWing 826
# Python solution

```

```

';
    cout << endl;
}

```

NQ130 模拟栈 AcWing 828

实现一个栈，栈初始为空，支持四种操作：(1) “push x” — 向栈顶插入一个数x；(2) “pop” — 从栈顶弹出一个数；(3) “empty” — 判断栈是否为空；(4) “query” — 查询栈顶元素。现在要对栈进行M个操作，其中的每个操作3和操作4都要输出相应的结果。 $1 \leq M \leq 100000$, $1 \leq x \leq 10^9$ 所有操作保证合法。

输入 第一行包含整数M，表示操作次数。接下来M行，每行包含一个操作命令，操作命令为“push x”，“pop”，“empty”，“query”中的一种。

输出 对于每个“empty”和“query”操作都要输出一个查询结果，每个结果占一行。其中，“empty”操作的查询结果为“YES”或“NO”，“query”操作的查询结果为一个整数，表示栈顶元素的值。

来源 AcWing 828

输入	输出
10	5
push 5	5
query	YES
push 6	4
pop	NO
query	
pop	
empty	
push 4	
query	
empty	

提示 原题

C++

```
#include <iostream>

using namespace std;
const int N = 100010;

int m;
int stk[N], tt; // 数组模拟栈

int main()
{
    cin >> m;
    while (m --)
    {
        string op;
        int x;

        cin >> op;
        if (op == "push") // 栈顶指针+1, 把数压入数组
        {
            cin >> x;
            stk[++tt] = x; // 压入一个元素
        }
        else if (op == "pop") tt --; // 修改栈顶指针即可
        else if (op == "empty") cout << (tt? "NO": "YES") << endl; // tt>0表示非空
        else cout << stk[tt] << endl; // 返回栈顶
    }
}
```

Python

```
# AcWing 828
# Python solution
```

NQ131 模拟队列 AcWing 829

实现一个队列，队列初始为空，支持四种操作：(1) “push x” — 向队尾插入一个数x；(2) “pop” — 从队头弹出一个数；(3) “empty” — 判断队列是否为空；(4) “query” — 查询队头元素。现在要对队列进行M个操作，其中的每个操作3和操作4都要输出相应的结果。1≤M≤100000, 1≤x≤10⁹, 所有操作保证合法。

输入 第一行包含整数M，表示操作次数。接下来M行，每行包含一个操作命令，操作命令为“push x”，“pop”，“empty”，“query”中的一种。

输出 对于每个“empty”和“query”操作都要输出一个查询结果，每个结果占一行。其中，“empty”操作的查询结果为“YES”或“NO”，“query”操作的查询结果为一个整数，表示队头元素的值。

来源 AcWing 829

输入	输出
10	NO
push 6	6
empty	YES
query	4
pop	
empty	
push 3	
push 4	
pop	

```
query
push 6
```

提示 原题

C++

```
#include <iostream>
using namespace std;
const int N = 1000007;
int q[N], hh, tt=-1;

int main(){
    int m;
    cin>>m;
    while (m--){
        int x;

        string op;
        cin>>op;

        if (op == "push") {
            //(1) "push x" - 向队尾插入一个数x;
            cin>>x;
            q[++tt] = x;
        } else if (op == "pop") // (2) "pop" - 从队头弹出一个数;
            hh++;
        else if (op == "empty") // (3) "empty" - 判断队列是否
为 空;
            cout<<(hh==tt?"NO":"YES")<<endl;
        else cout<<q[tt]<<endl; // (4) "query" - 查询队头元素。
    }

    return 0;
}
```

Python

```
# AcWing 829
# Python solution
```

NQ132 表达式求值 AcWing 3302

给定一个表达式，其中运算符仅包含+,-,*,/（加减乘整除），可能包含括号，请你求出表达式的最终值。题目保证表达式合法。整除向0取整（C++/Java默认行为，Python的//是向下取整不能直接用）。

- 输入** 共一行，为给定表达式。
- 输出** 共一行，为表达式的结果。
- 来源** AcWing 3302

输入

(2+2)*(1+1)

输出

8

提示 数据范围: 表达式长度不超过 10^5

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>
#include <stack>
#include <unordered_map>

using namespace std;

stack<int> num; //存储计算过程中的数字
stack<char> op; //存储操作符

void eval()
{
    auto b = num.top(); num.pop(); //第二个数
    auto a = num.top(); num.pop(); //第一个数
    auto c = op.top(); op.pop();

    int x;
    if (c == '+') x = a + b;
    else if (c == '-') x = a - b;
    else if (c == '*') x = a * b;
    else x = a / b;
    num.push(x);
}

int main()
{
    //优先级, 可扩展为其他符号的优先级
    unordered_map<char, int> pr{{'+', 1}, {'-', 1}, {'*', 2}, {'/', 2}};
    string str; //输入都存储在字符串内
    cin >> str;
    for (int i = 0; i < str.size(); i++)
    {
        auto c = str[i]; //读入每个char
        if (isdigit(c)) //判断是否是数字
        {
            int x = 0, j = i;
            while (j < str.size() && isdigit(str[j]))
                x = x * 10 + str[j++] - '0';
            i = j - 1; //调整i的位置
            num.push(x); //压栈
        }
        else if (c == '(') op.push(c);
        else if (c == ')')
        {
            //把前面所有的表达式都出栈运算
            while (op.top() != '(') eval();
            op.pop();
        }
        else
        {
            //根据操作符的优先级执行运算
            while (op.size() && op.top() != '(' && pr[op.top()]
                >= pr[c])
            {
                eval();
                op.pop();
            }
            op.push(c);
        }
    }
    //处理剩余的操作符
    while(op.size()) eval();
    cout<<num.top() <<endl;
    return 0;
}

```

Python

```

# AcWing 3302
# Python solution

```

NQ133 单调栈 AcWing 830

给定一个长度为N的整数数列，输出每个数左边第一个比它小的数，如果不存在则输出-1。 $1 \leq N \leq 10^5$ $1 \leq \text{数列中元素} \leq 10^9$

- 输入** 第一行包含整数N，表示数列长度。 第二行包含N个整数，表示整数数列。
- 输出** 共一行，包含N个整数，其中第i个数表示第i个数的左边第一个比它小的数，如果不存在则输出-1。
- 来源** AcWing 830

输入	输出
5	-1 3 -1 2 2
3 4 2 7 5	

提示 [原题链接](#) [参考题解](#) [Y总讲解](#) [Y总代码](#)

C++

```
#include <iostream>
using namespace std;
const int N=100007;
int n;
int stk[N],tt;//数组模拟栈实现单调栈
int main ()
{
    cin>> n;
    for (int i=0; i< n; i++){
        int x; cin>>x;
        while (tt && stk[tt]>= x)//tt为空，并且单调栈顶元素比
x大
            tt--;//出栈，这些数永远不会用到
        if (tt) cout<<stk[tt]<<' '; //栈顶就是比x小的元素，
直接输出
        else cout<<-1<<' '; //x左边没有任何数比它小
        //入栈
        stk[++tt] = x;
    }
    return 0;
}
```

Python

```
# AcWing 830
# Python solution
```

NQ134 滑动窗口 AcWing 154

给定一个大小为 $n \leq 10^6$ 的数组。有一个大小为k的滑动窗口，它从数组的最左边移动到最右边。您只能在窗口中看到k个数字。每次滑动窗口向右移动一个位置。以下是一个例子：该数组为[1 3 -1 -3 5 3 6 7]，k为3。您的任务是确定滑动窗口位于每个位置时，窗口中的最大值和最小值。

- 输入** 输入包含两行。第一行包含两个整数n和k，分别代表数组长度和滑动窗口的长度。第二行有n个整数，代表数组的具体数值。同行数据之间用空格隔开。

输出

输出包含两个。第一行输出，从左至右，每个位置滑动窗口中的最小值。第二行输出，从左至右，每个位置滑动窗口中的最大值。

来源

AcWing 154

输入

```
8 3
1 3 -1 -3 5 3 6 7
```

输出

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

提示 [原题链接](#) [参考题解](#) [Y总讲解](#) [Y总代码](#)

C++

```

#include <iostream>
using namespace std;

const int N=10000007;

int n,k;
int a[N],q[N]; //数组模拟队列
int hh=0,tt=-1; //hh队头,tt对尾

void initQueue(){
    hh=0; tt=-1;
}

int main()
{
    scanf("%d%d",&n,&k);
    for (int i=0; i<n; i++) scanf("%d",&a[i]); //读入数字

    //求滑动窗口内的最小值
    initQueue(); //初始化队列, 队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front 删除队头

        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt&& a[q[tt]]>= a[i]) tt--; //pop back 删除队尾

        q[++tt] =i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最小值
    }
    puts("");

    //求滑动窗口内的最大值
    initQueue(); //初始化队列, 队列存储的是数组下标
    for (int i=0; i<n; i++)
    {
        //判断队头是否已经滑出窗口
        if (hh<=tt && i-k+1> q[hh]) hh++; //pop front 删除队头

        //!单调队列, 去掉所有降序的队尾点
        while (hh<=tt&& a[q[tt]]<=a[i]) tt--; //pop back 删除队尾

        q[++tt] =i; //把当前i值入队列
        if (i>=k-1) //i在窗口内
            printf("%d ",a[q[hh]]); //队头就是最大值
    }
    puts("");
}

```

Python

```

# AcWing 154
# Python solution

```

NQ135 KMP字符串 AcWing 831

给定一个模式串S，以及一个模板串P，所有字符串中只包含大小写英文字母以及阿拉伯数字。模板串P在模式串S中多次作为子串出现。求出模板串P在模式串S中所有出现的位置的起始下标。数据范围： $1 \leq N \leq 10^5$
 $1 \leq M \leq 10^6$

输入

第一行输入整数N，表示字符串P的长度。第二行输入字符串P。第三行输入整数M，表示字符串S的长度。第四行输入字符串S。

输出

共一行，输出所有出现位置的起始下标（下标从0开始计数），整数之间用空格隔开。

来源

AcWing 831

输入	输出
3	0 2
aba	
5	
ababa	

提示 [原题链接](#) [Y总讲解](#) [参考题解](#) [Y总代码](#) [B站KMP讲解](#) [董晓讲解](#)

C++

```

#include <iostream>
using namespace std;
const int N = 100007, M = 1000007;
int n, m;          //n为待匹配字符串长度
char p[N], s[M];   //p为模板,s搜索范围
int ne[N];         // next数组,初始值为0

int main()
{
    // 第一行输入整数N, 表示字符串P的长度。
    // 第二行输入字符串P。
    // 第三行输入整数M, 表示字符串S的长度。
    // 第四行输入字符串S。
    cin >> n >> p + 1 >> m >> s + 1; //字符串从1开始存储, ci
    n会在末尾加上\0

    // 求next的过程 next[1]=0;从第二个字符开始计算next
    for (int i = 2, j = 0; i <= n; i++) //预处理p串计算前缀
    后缀的值
    {
        //双指针操纵与匹配过程一致
        while (j && p[i] != p[j + 1])
            j = ne[j];
        if (p[i] == p[j + 1])
            j++;
        ne[i] = j; //找到i的回退点j
    }

    // KMP 匹配过程
    for (int si = 1, pj = 0; si <= m; si++) //si,pj为字符串
    指针, 从1开始算
    {
        while (pj && s[si] != p[pj + 1]) // 用p串pj+1字符
        与s串的si比较 (提前看一个字符)
            pj = ne[pj];          //遇到不匹配, 回退p
        j指针
        if (s[si] == p[pj + 1])
            pj++; //pj进一位
        // p串匹配成功
        if (pj == n)
        { //抵达p串的最后一个字符
            cout << si - n << " "; //输出出现位置的下标
            pj = ne[pj];          //回退, 预备下一次的查找
        }
    }
    return 0;
}

```

Python

```

# AcWing 831
# Python solution

```

NQ136 模拟堆 AcWing 839

维护一个集合，初始时集合为空，支持如下几种操作：“I x”，插入一个数x；“PM”，输出当前集合中的最小值；“DM”，删除当前集合中的最小值（数据保证此时的最小值唯一）；“D k”，删除第k个插入的数；“C k x”，修改第k个插入的数，将其变为x；现在要进行N次操作，对于所有第2个操作，输出当前集合的最小值。
 $1 \leq N \leq 10^5$ $-109 \leq x \leq 10^9$ 数据保证合法。

输入

第一行包含整数N。接下来N行，每行包含一个操作指令，操作指令为“I x”，“PM”，“DM”，“D k”或“C k x”中的一种。

输出

对于每个输出指令“PM”，输出一个结果，表示当前集合中的最小值。每个结果占一行。

来源

AcWing 839

输入	输出
8	-10
I -10	6
PM	
I -10	
D 1	
C 2 8	
I 6	
PM	
DM	

提示 [原题链接](#) [Y总讲解](#)

C++

```

#include <iostream>
#include <algorithm>
#include <string.h>

using namespace std;

const int N = 100007;

// hp是heap pointer的缩写，表示堆数组中下标到第k个插入的映射
// ph是pointer heap的缩写，表示第k个插入到堆数组中的下标的映射
// hp和ph数组是互为反函数的
int h[N], ph[N], hp[N], cnt;

void heap_swap(int a, int b)
{
    swap(ph[hp[a]], ph[hp[b]]); // 堆数组的下标
    swap(hp[a], hp[b]); // 下标与第几个插入元素的映射
    swap(h[a], h[b]); // 堆元素
}

void down(int u)
{
    int t = u;
    if (u*2 <= cnt && h[u*2] < h[t]) t = u*2; // u*2左儿子
    if (u*2+1 <= cnt && h[u*2+1] < h[t]) t = u*2+1; // u*2+1右儿子
    if (u != t)
    {
        heap_swap(u, t);
        down(t);
    }
}

void up(int u)
{
    while (u/2 && h[u] < h[u/2])
    {
        heap_swap(u, u/2);
        u >>= 1;
    }
}

int main()
{
    int n, m = 0;
    scanf("%d", &n);
    while(n --)
    {
        char op[5];
        int k, x;
        scanf("%s", op);
        // 在堆最后一个元素插入元素
        if (!strcmp(op, "I"))
        {
            scanf("%d", &x);
            cnt ++;
            m ++;
            ph[m] = cnt, hp[cnt] = m;
            h[cnt] = x;
            up(cnt); // 插入后执行up(cnt)
        }
        else if (!strcmp(op, "PM")) printf("%d\n", h[1]); // 堆顶是最小值
    }
}

```

Python

```

# AcWing 839
# Python solution

```

```
else if (!strcmp(op, "DM"))
{
    heap_swap(1, cnt);
    cnt --;
    down(1); //删除最小值后执行down(1)
}
else if (!strcmp(op, "D"))
{
    scanf("%d", &k); //删除第k个元素
    k = ph[k]; //取第k个元素下标
    heap_swap(k, cnt); //与最后一个元素交换
    cnt--;
    up(k);
    down(k);
}
else {
    scanf("%d%d", &k, &x); //修改第k个数
    k = ph[k];
    h[k] = x;
    up(k);
    down(k);
}
}
return 0;
}
```